Intro_data_preprocessing

October 25, 2025

1 Data Preprocessing

`Data preprocessing` is the essential stage in machine learning that ensures raw data is transformed into a clean, structured, and consistent format suitable for training reliable models. Since real-world data is often noisy, incomplete, and inconsistent, the preprocessing stage plays a key role in enhancing model accuracy, generalization, and interpretability.

The modern data preprocessing pipeline typically includes the following **11 key steps**:

- **Data Collection:** Acquiring relevant, high-quality datasets from appropriate sources to represent the problem.
- **Data Cleaning:** Detecting and correcting errors, inconsistencies, or duplicates; handling noise, irrelevant data, and anomalies to ensure data integrity.
- **Handling Missing Values:** Managing gaps in the dataset using techniques such as mean, median, or mode imputation, or more advanced methods like KNN or regression-based imputation.
- **Data Integration:** Merging data from multiple sources or formats (databases, APIs, flat files) into a unified dataset while ensuring consistency and removing redundancy.
- **Data Transformation:** Normalizing, standardizing, encoding categorical variables, and applying domain-specific conversions to make all features numerically consistent for modeling.
- **Feature Engineering:** Creating new or derived features (e.g., ratios or interaction terms) that reveal hidden relationships or improve predictive performance.
- **Outlier Detection and Treatment:** Identifying and mitigating the influence of extreme values using methods such as IQR capping or percentile winsorization.
- **Skewness Reduction / Log Transformation:** Reducing skew in feature distributions using transformations (e.g., log, Box-Cox) to stabilize variance and improve model convergence.
- **Multicollinearity Detection (Variance Inflation Factor - VIF):** Measuring correlation among features to remove redundant variables that can distort model interpretation.
- **Handling Imbalanced Data:** Applying resampling methods like SMOTE or undersampling to balance the target variable distribution and improve classification fairness.
- **Data Reduction and Preprocessing Pipelines:** Using dimensionality reduction (e.g., PCA or feature selection) and automated `Pipeline` workflows for streamlined preprocessing, ensuring reproducibility and efficient deployment.

These steps collectively ensure that the dataset is high-quality, balanced, and feature-ready, ultimately enhancing both the performance and reliability of any machine learning model.

[12]:
```
# Suppressing Warnings
import warnings

# Ignore FutureWarnings
warnings.simplefilter(action='ignore', category=FutureWarning)

# Ignore RuntimeWarnings
warnings.simplefilter(action='ignore', category=RuntimeWarning)
```

1.1 Implementing The Data Preprocessing Concepts on a Synthetic Dataset

- Let's create a synthetic dataset and walk through all the data preprocessing steps with code and explanations at each step. This will consolidate your understanding with practical implementation.

Step 1: Synthetic Dataset Creation - We generate a dataset with numerical and categorical variables, purposely inserting missing values and outliers to simulate real-world messy data.

[13]:
```
import numpy as np
import pandas as pd

# Fix random seed for reproducibility
np.random.seed(42)
size = 1000

data = {
    'Age': np.random.randint(18, 70, size),
    'Annual Income (k$)': np.random.normal(60, 15, size),
    'Membership Years': np.random.randint(1, 20, size),
    'Credit Score': np.random.normal(650, 70, size),
    'Account Balance': np.random.normal(20000, 5000, size),
    'Gender': np.random.choice(['Male', 'Female'], size),
    'City': np.random.choice(['CityA', 'CityB', 'CityC'], size),
    'Purchased': np.random.choice(['Yes', 'No'], size, p=[0.3, 0.7])
}

df = pd.DataFrame(data)

# Introduce missing values intentionally
missing_indices = np.random.choice(df.index, 50, replace=False)
df.loc[missing_indices, 'Annual Income (k$)'] = np.nan

# Introduce outliers intentionally
outliers_idx = np.random.choice(df.index, 10, replace=False)
df.loc[outliers_idx, 'Account Balance'] *= 5
```

```
# Add correlated feature to create multicollinearity
noise = np.random.normal(0, 5, size)
df['Income Proxy'] = df['Membership Years'] * 3 + noise

print(df.head())
```

```
   Age  Annual Income (k$)  Membership Years  Credit Score  Account Balance  \
0   56           35.903305                 5    573.764392     22582.586422
1   69           63.051955                 4    639.092497     24864.982727
2   46           48.654739                15    652.650334     90612.652841
3   32           38.666194                 4    690.920084     25911.764220
4   60           50.301407                10    650.266215     22408.918539

   Gender   City Purchased  Income Proxy
0    Male  CityC        No     20.049777
1    Male  CityB        No      5.341909
2  Female  CityC        No     39.905317
3  Female  CityC        No      3.974243
4  Female  CityB       Yes     25.870999
```

Explanation: We simulate numeric and categorical features, add missing entries, outliers, and a correlated column to create realistic preprocessing challenges.

Step 2: Data Cleaning - Handling Missing Values - Detect missing values and impute with the column mean.

[14]:
```
from sklearn.impute import SimpleImputer

print("Missing values before imputation:\n", df.isnull().sum())

imputer = SimpleImputer(strategy='mean')
df['Annual Income (k$)'] = imputer.fit_transform(df[['Annual Income (k$)']])

print("Missing values after imputation:\n", df.isnull().sum())
```

```
Missing values before imputation:
 Age                    0
Annual Income (k$)     50
Membership Years        0
Credit Score            0
Account Balance         0
Gender                  0
City                    0
Purchased               0
Income Proxy            0
dtype: int64
Missing values after imputation:
 Age                    0
```

```
Annual Income (k$)    0
Membership Years      0
Credit Score          0
Account Balance       0
Gender                0
City                  0
Purchased             0
Income Proxy          0
dtype: int64
```

Explanation: Handling missing values prevents issues during modeling; mean imputation fills gaps using average income.

Step 3: Data Integration - This synthetic dataset is already integrated, so no extra action is required here.

Step 4: Data Transformation - Encoding and Scaling - Build pipelines to encode categorical features and scale numeric ones.

[15]:
```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

num_cols = ['Age', 'Annual Income (k$)', 'Membership Years', 'Credit Score',␣
 ↪'Account Balance', 'Income Proxy']
cat_cols = ['Gender', 'City']

num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])

cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', drop='first'))
])

preprocessor = ColumnTransformer([
    ('num', num_pipeline, num_cols),
    ('cat', cat_pipeline, cat_cols)
])

X = df.drop(columns=['Purchased'])
y = df['Purchased']

X_transformed = preprocessor.fit_transform(X)

print("Data transformation done.")
```

```
Data transformation done.
```

Explanation: Scaling numeric features helps algorithms converge; encoding transforms categories to numeric form.

Step 5: Feature Engineering - Create a derived feature representing income per year of membership (handling divide-by-zero).

[16]:
```
df['Income_per_Year'] = df['Annual Income (k$)'] / df['Membership Years'].
 ↪replace(0, 1)
print(df[['Annual Income (k$)', 'Membership Years', 'Income_per_Year']].head())
```

```
   Annual Income (k$)  Membership Years  Income_per_Year
0           35.903305                 5         7.180661
1           63.051955                 4        15.762989
2           48.654739                15         3.243649
3           38.666194                 4         9.666549
4           50.301407                10         5.030141
```

Explanation: New meaningful features can improve model insight and performance.

Step 6: Outlier Detection and Treatment using Interquartile Range (IQR) Cap extreme Account Balance values.

[17]:
```
Q1 = df['Account Balance'].quantile(0.25)
Q3 = df['Account Balance'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

df['Account Balance'] = np.where(df['Account Balance'] > upper_bound,␣
 ↪upper_bound,
                                 np.where(df['Account Balance'] < lower_bound,␣
 ↪lower_bound, df['Account Balance']))
print("Outliers capped using IQR method.")
```

```
Outliers capped using IQR method.
```

Explanation: Reducing outlier impact ensures model robustness.

Step 7: Skewness Reduction - Apply log transformation to skewed numeric features.

[18]:
```
skewed_cols = ['Annual Income (k$)', 'Account Balance']
for col in skewed_cols:
    df[col] = np.log1p(df[col].clip(lower=0))  # avoid log of negative values

print("Log transformation applied to reduce skewness.")
```

```
Log transformation applied to reduce skewness.
```

Explanation: Normalizing distributions improves model stability and predictions.

Step 8: Multicollinearity Detection with Variance Inflation Factor (VIF) - Identify features highly correlated with others.

[19]:
```
from statsmodels.stats.outliers_influence import variance_inflation_factor

X_vif = df[num_cols + ['Income_per_Year']].copy()

# Replace infinite values and impute missing before VIF
X_vif.replace([np.inf, -np.inf], np.nan, inplace=True)
X_vif.fillna(X_vif.mean(), inplace=True)

vif = pd.DataFrame()
vif['Feature'] = X_vif.columns
vif['VIF'] = [variance_inflation_factor(X_vif.values, i) for i in range(X_vif.
 ↪shape[1])]

print(vif)
```

```
              Feature         VIF
0                 Age    9.551975
1  Annual Income (k$)  232.702907
2    Membership Years   52.026000
3        Credit Score   88.325438
4     Account Balance  269.729841
5        Income Proxy   44.620573
6     Income_per_Year    3.331559
```

Explanation: Detecting multicollinearity guides feature selection and prevents unreliable coefficient estimates.

Step 9:Handling Imbalanced Data with Random Oversampling - Random oversampling is a simple technique that balances the dataset by duplicating existing samples from the minority class until the class sizes are equal.

[20]:
```
from sklearn.utils import resample
import pandas as pd

# Map target to binary labels
y_binary = df['Purchased'].map({'Yes': 1, 'No': 0})

# Apply preprocessing pipeline on features
X_transformed = preprocessor.fit_transform(df.drop(columns=['Purchased']))

# Combine features and target into one DataFrame for resampling
df_balancer = pd.concat([pd.DataFrame(X_transformed), y_binary.
 ↪reset_index(drop=True)], axis=1)

# Separate majority and minority classes
df_majority = df_balancer[df_balancer[y_binary.name] == 0]
```

```
df_minority = df_balancer[df_balancer[y_binary.name] == 1]

# Upsample minority class by random repetition
df_minority_upsampled = resample(df_minority,
                                 replace=True,
                                 n_samples=len(df_majority),
                                 random_state=42)

# Combine majority and upsampled minority class
df_upsampled = pd.concat([df_majority, df_minority_upsampled])

# Extract features and target
X_res = df_upsampled.drop(columns=[y_binary.name]).values
y_res = df_upsampled[y_binary.name].values

# Display distributions
print(f"Original class distribution:\n{y_binary.value_counts()}")
print(f"Balanced class distribution after Random Oversampling:\n{pd.
  ↪Series(y_res).value_counts()}")
```

```
Original class distribution:
Purchased
0    685
1    315
Name: count, dtype: int64
Balanced class distribution after Random Oversampling:
0    685
1    685
Name: count, dtype: int64
```

Explanation: This approach increases the minority class size by replication rather than synthetic generation, helping classifiers learn better decision boundaries without introducing artificial variation.

Step 10: Data Splitting - Split the balanced dataset into training and testing sets.

[21]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X_res, y_res, test_size=0.
  ↪2, random_state=42)
print(f"Training size: {X_train.shape}, Testing size: {X_test.shape}")
```

```
Training size: (1096, 9), Testing size: (274, 9)
```

Step 11: Pipeline Automation

- Each transformation is encapsulated in pipelines ensuring consistent application and preventing data leakage.

[22]:
```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder,␣
 ↪FunctionTransformer
from sklearn.impute import SimpleImputer
import numpy as np

# Define numerical and categorical columns
num_cols = ['Age', 'Annual Income (k$)', 'Membership Years', 'Credit Score',␣
 ↪'Account Balance', 'Income Proxy', 'Income_per_Year']
cat_cols = ['Gender', 'City']

# Function to clip outliers based on IQR
def cap_outliers(X):
    # Assume X is a 2D numpy array for numerical columns
    X = X.copy()
    for i in range(X.shape[1]):
        column = X[:, i]
        Q1 = np.percentile(column, 25)
        Q3 = np.percentile(column, 75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        column = np.clip(column, lower, upper)
        X[:, i] = column
    return X

# Function to apply log1p transformation to reduce skewness
log_transformer = FunctionTransformer(np.log1p, validate=True)

# Numerical pipeline with all steps: missing value imputation, outlier capping,␣
 ↪log transform and scaling
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),      # Handle missing values
    ('outlier_cap', FunctionTransformer(cap_outliers, validate=False)),  # Clip␣
 ↪outliers
    ('log_transform', log_transformer),               # Reduce skewness
    ('scaler', StandardScaler())                      # Scale features
])

# Categorical pipeline: missing value imputation + one hot encoding
cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', drop='first'))
])

# Full preprocessing pipeline
```

```
preprocessing_pipeline = ColumnTransformer([
    ('num', num_pipeline, num_cols),
    ('cat', cat_pipeline, cat_cols)
])

# Fit and transform with pipeline
X_processed = preprocessing_pipeline.fit_transform(df.
 ↪drop(columns=['Purchased']))

print("All numerical and categorical preprocessing steps completed in pipeline.
 ↪")
```

```
All numerical and categorical preprocessing steps completed in pipeline.
```

-

1) We observed that some columns contained missing values, which were identified using the .isnull().sum() function.
2) Missing values were handled using suitable techniques such as mean, median, or mode imputation, depending on the column type.
3) Some columns contained categorical data, which we converted into numerical format using label encoding or one-hot encoding to make it suitable for machine learning models.
4) Duplicate records were checked and removed to ensure data consistency.
5) We examined each column's data type and made conversions where necessary (for example, changing object types to numeric).
6) Outliers were identified using boxplots and treated appropriately to reduce their impact on the analysis.
7) Continuous numerical features were standardized or normalized to bring all variables to a similar scale.
8) Feature selection or feature engineering was done to remove irrelevant attributes and improve model performance.
9) The cleaned dataset was verified using .info() and .describe() to ensure no missing or inconsistent values remained.